



RELEASEPILOT

A secure CI/CD pipeline for a cloud-native API

Docker • GitHub Actions • Amazon ECR • Amazon EKS • HTTPS

Deployment verified over HTTPS • AWS resources decommissioned after the demo

What ReleasePilot demonstrates

A small operational API designed to prove an end-to-end DevOps workflow.

Build

Containerized Node.js API with a non-root runtime user

Validate

Lint, tests, dependency audit, and Trivy image scan

Deploy

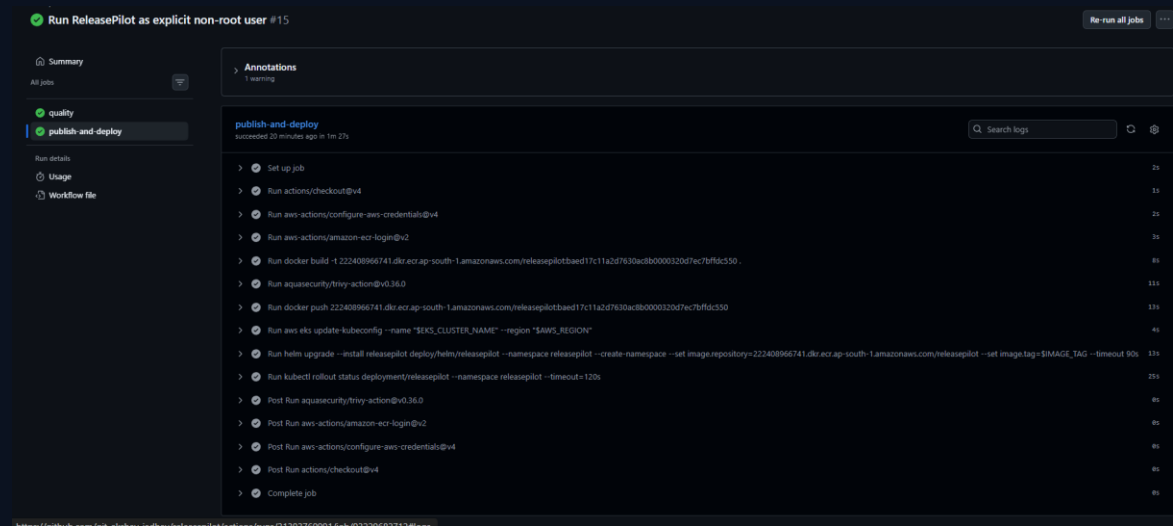
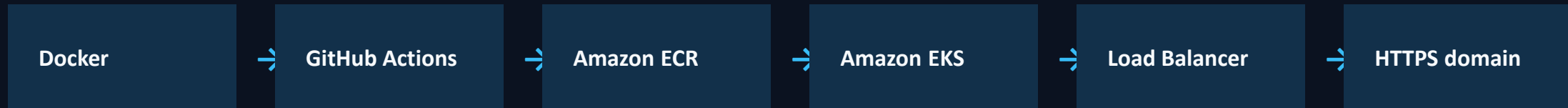
Push a versioned image to ECR and roll it out to EKS

Operate

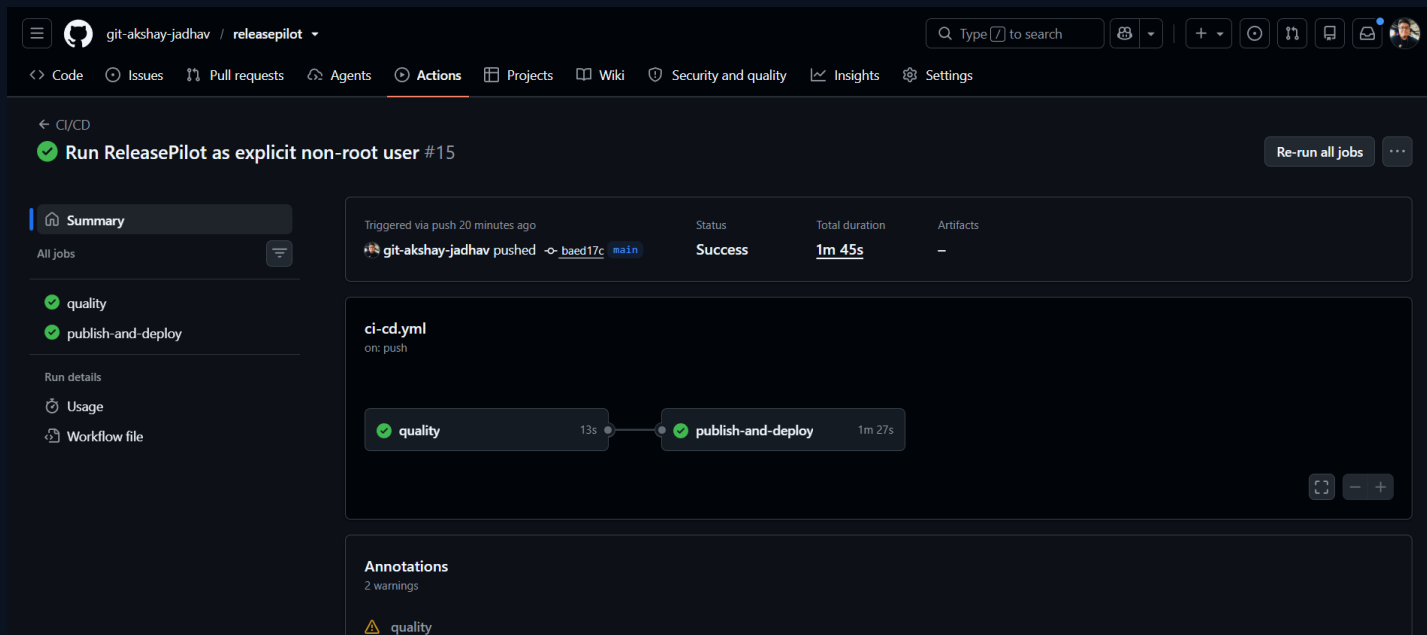
Health, readiness, and Prometheus metrics endpoints

The delivery path is automated

Every push to main moves through quality checks before deployment.



CI/CD validates and deploys the release



The screenshot shows the GitHub Actions interface for the 'releasepilot' repository. The pipeline is titled 'Run ReleasePilot as explicit non-root user #15' and is currently in a 'Success' state. The pipeline was triggered by a push to the 'main' branch by user 'git-akshay-jadhav' 20 minutes ago. The total duration of the pipeline is 1m 45s. The pipeline consists of two jobs: 'quality' (13s) and 'publish-and-deploy' (1m 27s). The 'quality' job is marked with a green checkmark, indicating it passed. The 'publish-and-deploy' job is also marked with a green checkmark, indicating it passed. The pipeline is defined in the 'ci-cd.yml' file, which is triggered on a push. The 'Annotations' section shows 2 warnings, with one warning from the 'quality' job.

git-akshay-jadhav / releasepilot

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

CI/CD

Run ReleasePilot as explicit non-root user #15

Summary

All jobs

quality

publish-and-deploy

Run details

Usage

Workflow file

Triggered via push 20 minutes ago

Status Success

Total duration 1m 45s

Artifacts -

ci-cd.yml

on: push

quality 13s

publish-and-deploy 1m 27s

Annotations

2 warnings

quality

Successful pipeline evidence

- quality checks passed
- AWS OIDC authentication
- image built and scanned
- image pushed to ECR
- Helm deployment applied
- Kubernetes rollout verified

AWS deployment evidence

EKS cluster

Amazon Elastic Kubernetes Service

Dashboard

Clusters

▼ Settings

▼ Amazon EKS Anywhere

▼ Related services

releasepilot-akshay

Cluster info

Status

Active

Kubernetes version

1.36

Support period

Standard support until August 2, 2027

Provider

EKS

Cluster health

0

Upgrade insights

5

Node health issues

0

Capability issues

0

Overview

Resources

Compute

Networking

Add-ons

Capabilities

Access

Observability

Update history & back

Private ECR image repository

Amazon ECR

Private registry

Repositories

Private repositories (1)

Repository name

URI

Created at

Tag immutability

Encryption type

releasepilot

222408966741.dkr.ecr.ap-south-1.amazonaws.com/releasepilot

August 09, 2026, 12:48:48 (UTC+05.5)

Mutable

AES-256

Listeners

Network mapping

Security

Health checks

Target instances

Monitoring

Attributes

Tags

Listeners

A Classic Load Balancer listener uses the protocols and ports you configure, to both check for connection requests and forward received traffic to instances. The listener uses its protocol and port to check for connection requests. When traffic is received by the listener, it's forwarded to registered EC2 instances using the instance protocol and port.

Filter listeners

Protocol:Port

Instance Pr...

Security policy

Default SSL/TLS certificate

Cookie stickiness

HTTPS:443

HTTP:31469

ELBSecurityPolicy-2016-08

app.demomine.online (Certificate ID: b5f02d7...

Not applicable

TCP:80

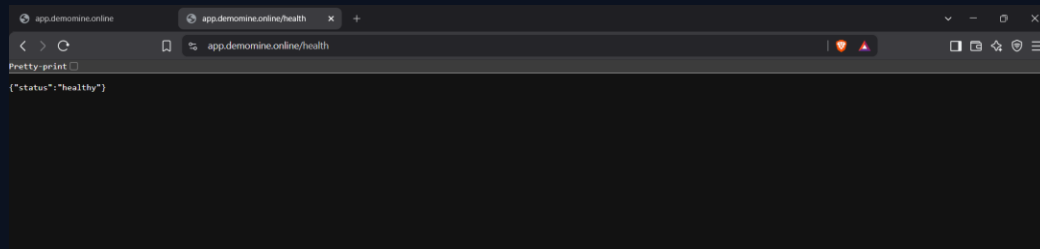
TCP:31469

Not applicable

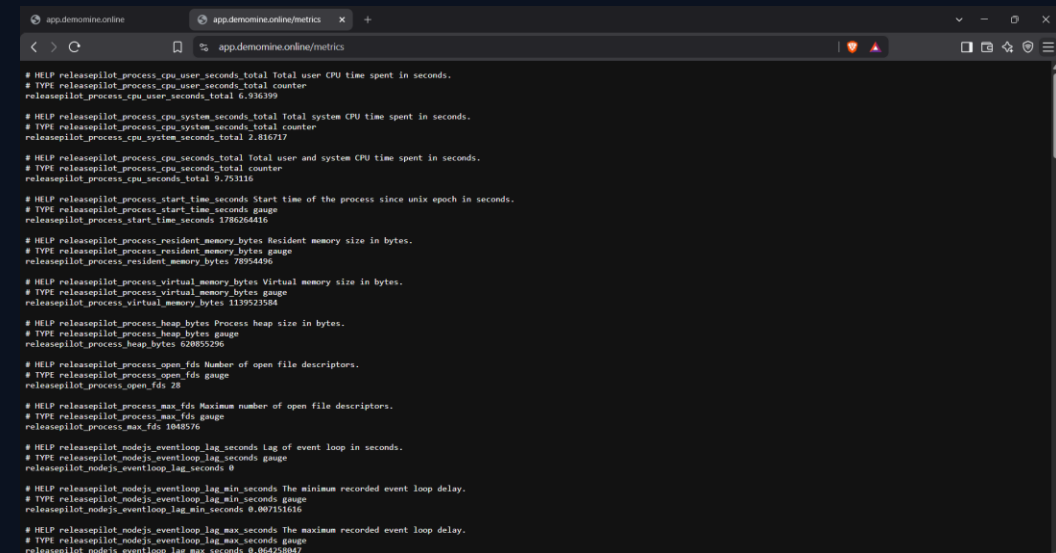
Not applicable

Not applicable

The application is observable and secure



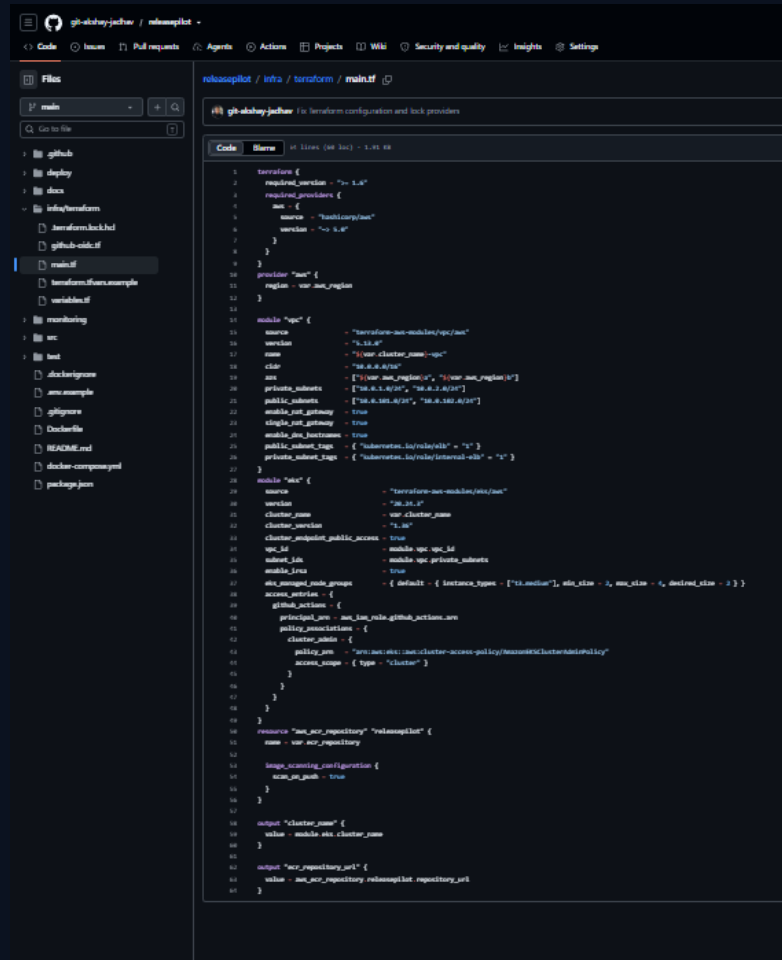
/health returns 200



/metrics exposes Prometheus metrics

HTTPS is terminated by the AWS Load Balancer using an ACM certificate for app.demomine.online.

Infrastructure is reproducible with Terraform



The screenshot shows a GitHub repository named 'gh-releases-jenkins' with a file named 'main.tf'. The file contains Terraform code for provisioning AWS infrastructure. The code is organized into modules for 'vpc', 'eks', and 'ecr'. The 'vpc' module creates a VPC, subnets, and a route table. The 'eks' module creates an EKS cluster, node group, and role. The 'ecr' module creates an ECR repository. The code also includes variables for region, vpc cidr, and other parameters.

```
1 terraform {
2   required_version = "~> 1.5"
3   required_providers {
4     aws = {
5       source  = "hashicorp/aws"
6       version = "~> 5.0"
7     }
8   }
9 }
10 provider "aws" {
11   region = var.aws_region
12 }
13 module "vpc" {
14   source      = "terraform-aws-modules/vpc/aws"
15   version     = "3.12.0"
16   name        = "vpc-cluster-private"
17   cidr        = "10.0.0.0/16"
18   azs         = ["${var.aws_region}${var.aws_az_prefix}"]
19   private_subnets = ["${var.aws_region}${var.aws_az_prefix}"]
20   public_subnets  = ["${var.aws_region}${var.aws_az_prefix}"]
21   enable_nat_gateway = true
22   single_nat_gateway = true
23   enable_vpn_gateway = true
24   private_subnet_tags = { "kubernetes.io/role/elastic" = "1" }
25   public_subnet_tags  = { "kubernetes.io/role/elb" = "1" }
26 }
27 module "eks" {
28   source      = "terraform-aws-modules/eks/aws"
29   version     = "16.35.0"
30   cluster_name = var.cluster_name
31   cluster_version = "1.16"
32   cluster_endpoint_public_access = true
33   vpc_id       = module.vpc.vpc_id
34   subnet_ids   = module.vpc.private_subnets
35   enable_iam_auth = true
36   aws_managed_prometheus_metrics = { default = { instance_types = ["t3.medium"], eks_iam_role = "eks_iam_role", desired_iam_role = "1" } }
37   github_actions = {
38     principal_arn = aws_iam_role.github_actions.arn
39     policy_name    = {
40       cluster_policy = {
41         policy_arn = "arn:aws:iam::aws:policy/managed/elasticloadbalancing"
42         access_type = { type = "cluster" }
43       }
44     }
45   }
46 }
47 resource "aws_ecr_repository" "releasepiilot" {
48   name = var.ecr_repository
49   image_scanning_configuration {
50     scan_on_push = true
51   }
52 }
53 output "cluster_name" {
54   value = module.eks.cluster_name
55 }
56 output "ecr_repository_arn" {
57   value = aws_ecr_repository.releasepiilot.repository_arn
58 }
```

Terraform provisions the VPC, EKS cluster, node group, ECR repository, and GitHub OIDC deployment role.

Outcome

A verified HTTPS deployment with automated CI/CD and observable health signals. Resources can be removed after the demo to control cost.